



Licenciatura en Sistemas

Ingeniería de Software III

Equipo Docente: Prof. Nicolás Pérez / Inst. Víctor Martínez

Trabajo Práctico: Análisis Estadístico de Chats de WhatsApp

Grupo 2:

- Thomas Livon, Desarrollador (Usuario Git: Thlivon)
 - Lautaro Medina, Desarrollador (Usuario Git: Yeshoa)
 - Gonzalo Boquete, QA Tester (Usuario Git: GonzaBoquete)
 - Malena Lescano, Analista (Usuario Git: MalenaLesc)
-

1. Introducción	2
1.1 Objetivo	2
1.2 Alcance	2
2. Arquitectura del Sistema	2
2.1 Módulos	2
2.1.1 Capa de Presentación (app.py)	2
2.1.2 Capa de Procesamiento (parser.py)	3
2.1.3 Capa de Análisis (analytics.py)	3
2.2 Diseño de Componentes	3
2.3 Diagrama de Componentes	4
3. Tecnologías Utilizadas	4
3.1 Python 3.13	4
3.2 Streamlit	4
3.3 Pandas	5
3.4 Matplotlib	5
3.5 WordCloud	5
3.6 Emoji	5
4. Diseño del Procesamiento	5
4.1 Lectura del archivo	5
4.2 Identificación del formato	5
4.3 Conversión de datos	5
5. Métricas Implementadas	5
5.1 Usuario más activo	5
5.2 Emoji más utilizado	6
5.3 Franja horaria más activa	6
5.4 Actividad por día de la semana	6
5.5 Ranking de actividad semanal	6
5.6 Frecuencia de palabras	6
5.7 Nube de palabras	6
6. Decisiones Técnicas	6
6.1 Uso de DataFrames	6
6.2 Exclusión de palabras irrelevantes	6
6.3 Soporte ZIP	6
7. Limitaciones	7
8. Metodología de Desarrollo y Gestión	7
8.1 Ciclo de Vida en V	7
8.2 Estrategia de Pruebas	7
8.3 Estrategia de Gestión: Gitflow	7
8.4 Gestión de Tareas	8
9. Conclusión	8

Documentación Técnica

1. Introducción

1.1 Objetivo

El presente documento describe la arquitectura, diseño e implementación del sistema "Analizador Estadístico de Chats de WhatsApp", desarrollado como trabajo práctico para la materia Ingeniería de Software III.

El sistema permite procesar exportaciones de conversaciones de WhatsApp en formatos TXT y ZIP, obteniendo métricas y visualizaciones relacionadas con la actividad de los participantes.

1.2 Alcance

La aplicación permite:

- Procesar chats exportados desde WhatsApp.
- Analizar la actividad de los participantes.
- Generar estadísticas descriptivas.
- Mostrar resultados mediante una interfaz gráfica desarrollada con Streamlit.

No se contempla el análisis de archivos multimedia.

2. Arquitectura del Sistema

2.1 Módulos

La solución se diseñó utilizando una arquitectura modular, permitiendo que la lógica de negocio sea independiente de la interfaz de usuario. Esta compuesta por tres componentes principales:

2.1.1 Capa de Presentación (app.py)

Aquí se definió la aplicación web interactiva de visualización y análisis de datos (Dashboard) construida con el framework Streamlit. Su propósito específico es procesar archivos de exportación de chats de WhatsApp (.txt o .zip) para generar métricas, gráficos estadísticos y una nube de palabras. Maneja el ciclo de vida del estado de la aplicación, el renderizado de gráficos y la interacción del usuario.

Responsable de:

- Interfaz gráfica de usuario.
- Carga de archivos.
- Gestión del estado de la aplicación.
- Presentación de resultados.

2.1.2 Capa de Procesamiento (parser.py)

Es el módulo encargado de la lectura, extracción, transformación y limpieza de los datos provenientes del archivo exportado por WhatsApp. Su responsabilidad principal es convertir datos no estructurados (un archivo de texto plano proveniente de WhatsApp) en una estructura de datos procesable (DataFrame de pandas) para el análisis estadístico.

Responsable de:

- Lectura de archivos TXT.
- Lectura de archivos ZIP.
- Conversión de mensajes a estructuras de datos procesables.

2.1.3 Capa de Análisis (analytics.py)

Constituye la capa de lógica de negocio y análisis estadístico de la aplicación. Su función es recibir el DataFrame ya procesado (limpio y estructurado) y realizar operaciones de agregación, filtrado y cálculo estadístico para extraer conclusiones del chat introducido.

Responsable de:

- Cálculo de métricas.
- Obtención de estadísticas.
- Generación de datos para visualizaciones.

2.2 Diseño de Componentes

2.2.1 Parser

Implementa un motor de búsqueda de estado para resolver mensajes multilínea, garantizando que el DataFrame final no tenga registros fragmentados.

2.2.2 Analytics

Utiliza un enfoque funcional donde cada métrica es una función pura que recibe un DataFrame y devuelve una estructura de datos, asegurando la trazabilidad del cálculo.

2.3 Diagrama de Componentes

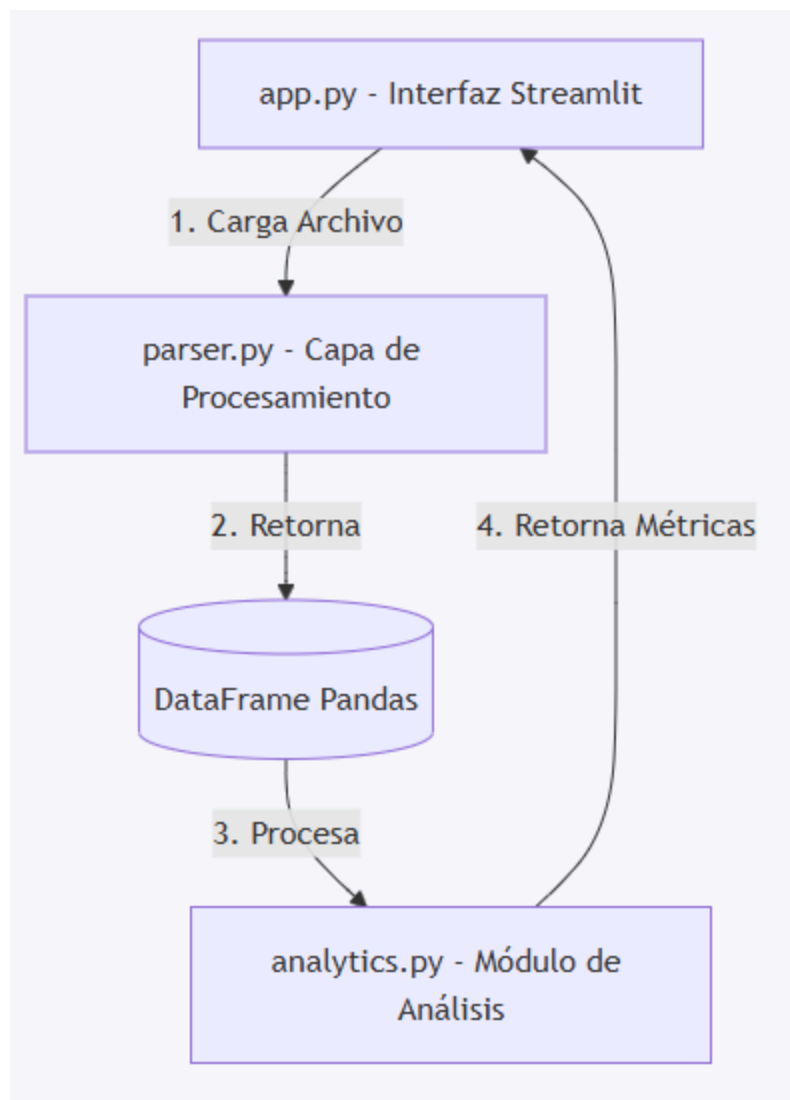


Figura 1. Diagrama de componentes del sistema.

El flujo comienza con la carga de un archivo por parte del usuario. El módulo parser transforma el archivo en un DataFrame estructurado, que luego es procesado por el módulo analytics para obtener métricas y estadísticas. Finalmente, los resultados son presentados al usuario mediante la interfaz desarrollada con Streamlit.

3. Tecnologías Utilizadas

3.1 Python 3.13

Lenguaje principal utilizado para el desarrollo.

3.2 Streamlit

Framework utilizado para la construcción de la interfaz web.

3.3 Pandas

Utilizado para la manipulación y análisis de datos.

3.4 Matplotlib

Generación de gráficos estadísticos.

3.5 WordCloud

Generación de nube de palabras.

3.6 Emoji

Procesamiento y normalización de emojis.

4. Diseño del Procesamiento

4.1 Lectura del archivo

La aplicación acepta:

- Archivos TXT exportados desde WhatsApp.
- Archivos ZIP que contienen exportaciones de WhatsApp.

4.2 Identificación del formato

Formato:

12/4/25, 21:03 - Usuario: Mensaje

4.3 Conversión de datos

Los mensajes son transformados a un DataFrame de Pandas con las siguientes columnas:

- fecha
- usuario
- mensaje

5. Métricas Implementadas

5.1 Usuario más activo

Determina el participante con mayor cantidad de mensajes enviados.

5.2 Emoji más utilizado

Cuenta la frecuencia de aparición de emojis y determina el más utilizado.

5.3 Franja horaria más activa

Agrupar mensajes por hora y determina el intervalo con mayor actividad.

5.4 Actividad por día de la semana

Calcula el porcentaje de mensajes enviados en cada día.

5.5 Ranking de actividad semanal

Ordena los días según el porcentaje de actividad registrado.

5.6 Frecuencia de palabras

Calcula la cantidad de apariciones de cada palabra excluyendo:

- Stopwords.
- Emojis.
- Nombres de participantes.

5.7 Nube de palabras

Representa visualmente las palabras más frecuentes del chat.

6. Decisiones Técnicas

6.1 Uso de DataFrames

Se eligió Pandas debido a su eficiencia para realizar operaciones de agrupamiento, filtrado y análisis estadístico.

6.2 Exclusión de palabras irrelevantes

Se implementó un conjunto de stopwords para mejorar la calidad del análisis de frecuencia de palabras.

6.3 Soporte ZIP

Se decidió incorporar procesamiento directo de archivos comprimidos para mejorar la experiencia del usuario.

7. Limitaciones

- No se procesan archivos multimedia.
- No se almacenan resultados de manera persistente.
- El análisis se realiza únicamente sobre mensajes exportados por WhatsApp.
- Solo se procesan chats exportados desde Android.

8. Metodología de Desarrollo y Gestión

8.1 Ciclo de Vida en V

Para el desarrollo de este sistema, se ha adoptado el **Modelo en V**, el cual garantiza una correspondencia estrecha entre las fases de diseño y las fases de validación.

- **Lado izquierdo (Definición):** Se definieron los requisitos funcionales (análisis de chats, métricas estadísticas) y la arquitectura modular (separación en módulos app, parser y analytics).
- **Lado derecho (Verificación y Validación):** Cada módulo desarrollado cuenta con pruebas que validan la correcta lógica de procesamiento y la coherencia de los resultados generados en la capa de análisis.

8.2 Estrategia de Pruebas

Durante el desarrollo se realizaron pruebas funcionales sobre cada una de las métricas implementadas y sobre el módulo de procesamiento de archivos.

Las pruebas incluyeron:

- Lectura de archivos TXT exportados desde WhatsApp.
- Lectura de archivos ZIP exportados desde WhatsApp.
- Validación del cálculo del usuario más activo.
- Validación del cálculo del emoji más utilizado.
- Validación de actividad por horario y día de la semana.
- Validación de frecuencia de palabras y generación de nube de palabras.

Los archivos utilizados para las pruebas y los resultados obtenidos se encuentran almacenados en la carpeta:

“tests/”

permitiendo reproducir y verificar el comportamiento de cada funcionalidad implementada.

8.3 Estrategia de Gestión: Gitflow

El proyecto utiliza **Gitflow** como flujo de trabajo para garantizar un historial de commits consistente y una gestión ordenada de funcionalidades:

- **main:** Rama destinada exclusivamente al código estable y funcional.

- **develop**: Rama principal de integración donde se consolidan los nuevos cambios.
- **feature/***: Ramas utilizadas para el desarrollo de funcionalidades específicas
- **fix/***: ramas destinadas a corrección de errores detectados durante el desarrollo o las pruebas.
- **Convenciones de Commits**: Se utiliza la convención *Conventional Commits* para facilitar la trazabilidad:
 - feat: para nuevas funcionalidades.
 - fix: para correcciones de errores.
 - docs: para cambios en la documentación.

8.4 Gestión de Tareas

Para la planificación y seguimiento de actividades se utilizó Trello como herramienta de gestión visual de tareas. El tablero nos permitió organizar el trabajo mediante tarjetas asociadas a funcionalidades, correcciones y actividades de documentación.

Cada tarea fue asignada a un integrante del equipo, facilitando el seguimiento del avance y la coordinación del trabajo colaborativo. La herramienta también permitió registrar el estado de cada actividad mediante columnas de trabajo (Pendiente, En Curso, Finalizada), mejorando la visibilidad del proyecto.

9. Conclusión

La solución desarrollada permite analizar conversaciones de WhatsApp de forma sencilla e intuitiva, proporcionando estadísticas relevantes mediante una interfaz web amigable y simple, utilizando un diseño modular que facilita su mantenimiento y evolución.